

INTRODUCTION TO NEURAL AND DEEP LEARNING

BACHELOR OF TECHNOLOGY (CSE SEC-C)

Semester-VI

to

K.R Mangalam University

By

AMAN

2301010180

Under the supervision of

Faculty Mentor:- Mrs Lucky Verma



Department of Computer Science and Engineering
School of Engineering and Technology
K.R Mangalam University, Gurugram- 122103, India

INDEX

Sr. No.	Title	Date	Signature
1	Introduction to Python & Numerical Computing using NumPy Array Operations and Mathematical Functions		
2	Pandas Series and DataFrame Operations for Data Manipulation, Sorting and Statistical Analysis		
3	Utilizing Basic SciPy Functions for Mathematical Optimization and Data Interpolation Techniques		
4	Introduction to Matplotlib Visualizations — Line Plots, Scatter Plots, Subplots and Graph Formatting		
5	Perceptron Learning Algorithm — Building a Single-Layer Perceptron to Learn the AND Logic Gate		
6	Limitation of Single-Layer Perceptron — Demonstrating the Failure to Solve the Non-Linear XOR Problem		
7	Visualizing Neural Network Activation Functions — Sigmoid, Tanh, ReLU and Leaky ReLU using Matplotlib		
8	Neural Network Classification using Scikit-Learn MLPClassifier with Training Loop and Accuracy Evaluation		
9	Comparing Optimization Algorithms — Standard Gradient Descent vs Adaptive Adam Optimizer		
10	Implementation of Recurrent Neural Networks and Convolutional Neural Networks using PyTorch		
11	Hopfield Network Implementation — Pattern Memorization, Noise Injection and Associative Memory Retrieval		

Experiment 1

Aim: To familiarize students with NumPy array operations and mathematical functions.

Code:

```
import numpy as np

# Experiment 1: Introduction to Python & Numerical Computing
# Aim: To familiarize students with NumPy array operations and mathematical
functions.

print("--- 1. Array Creation Functions ---")
array_1d = np.array([1, 2, 3, 4, 5])      # Convert a list to a 1D array
zeros_array = np.zeros((2, 3))           # Create a 2x3 matrix filled with
zeros                                     # zeros
ones_array = np.ones((2, 3))              # Create a 2x3 matrix filled with ones
random_array = np.random.rand(2, 2)      # Create a 2x2 matrix with
random values                             # random values
sequence = np.arange(0, 10, 2)            # Sequence from 0 to 10 with a
step of 2
spaced = np.linspace(0, 1, 5)             # 5 evenly spaced numbers from 0 to
1

print("1D Array:", array_1d)
print("Arange Sequence:", sequence)
print("Linspace:", spaced)

print("\n--- 2. Shape and Array Manipulation ---")
array_2d = np.array([[1, 2, 3], [4, 5, 6]])
print("Original 2D Array Shape:", array_2d.shape)
reshaped = np.reshape(array_1d, (5, 1))  # Reshape the 1D array into a
5x1 column
print("Reshaped 1D to (5,1):\n", reshaped)
transposed = np.transpose(array_2d)       # Swap rows and columns
print("Transposed 2D Array:\n", transposed)
```

```

print("\n--- 3. Mathematical Operations ---")
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print("Matrix Addition:\n", np.add(A, B))      # Element-wise addition
print("Matrix Subtraction:\n", np.subtract(A, B)) # Element-wise subtraction
print("Matrix Multiplication (Element-wise):\n", np.multiply(A, B))
print("Dot Product (True matrix multiplication):\n", np.dot(A, B))

print("\n--- 4. Statistical & Aggregation Functions ---")
data = np.array([15, 20, 35, 40, 50])
print("Max value:", np.max(data))
print("Min value:", np.min(data))
print("Mean (Average):", np.mean(data))
print("Sum of elements:", np.sum(data))
print("Standard Deviation:", np.std(data).round(2))

```

OUTPUT:

```

aman python3 exp1.py
--- 1. Array Creation Functions ---
1D Array: [1 2 3 4 5]
Arange Sequence: [0 2 4 6 8]
Linspace: [0.  0.25 0.5  0.75 1.  ]

--- 2. Shape and Array Manipulation ---
Original 2D Array Shape: (2, 3)
Reshaped 1D to (5,1):
[[1]
 [2]
 [3]
 [4]
 [5]]
Transposed 2D Array:
[[1 4]
 [2 5]
 [3 6]]

--- 3. Mathematical Operations ---
Matrix Addition:
[[ 6  8]
 [10 12]]
Matrix Subtraction:
[[-4 -4]
 [-4 -4]]
Matrix Multiplication (Element-wise):
[[ 5 12]
 [21 32]]
Dot Product (True matrix multiplication):
[[19 22]
 [43 50]]

--- 4. Statistical & Aggregation Functions ---
Max value: 50
Min value: 15
Mean (Average): 32.0
Sum of elements: 160
Standard Deviation: 12.88

```

Experiment 2

Aim: To demonstrate core operations on Pandas Series and DataFrames.

Code:

```
import pandas as pd

# Experiment 2: Pandas Series and DataFrame Operations
# Aim: To demonstrate core operations on Pandas Series and DataFrames.

print("--- 1. Pandas Series Operations ---")
# Create a basic Pandas Series with custom index letters
series_data = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'],
name="Scores")
print("Original Series:\n", series_data)

# Test out basic Series functionalities
print("\nMaximum Value in Series:", series_data.max())
print("Mean (Average) of Series:", series_data.mean())
print("Filtering Series (values > 25):\n", series_data[series_data > 25])

print("\n--- 2. Pandas DataFrame Operations ---")
# Create a DataFrame using a standard python dictionary
data_dict = {
    "Student": ["Alice", "Bob", "Charlie", "David"],
    "Age": [24, 27, 22, 32],
    "Grade": [88.5, 92.0, 79.5, 95.0]
}
df = pd.DataFrame(data_dict)
print("Original DataFrame:\n", df)

# Demonstrate basic DataFrame inspection
print("\nDataFrame Dimensions (Shape):", df.shape)
print("\nFetching the top 2 rows using .head():\n", df.head(2))

# Extract a single column (this returns a Series object automatically)
```

```
extracted_series = df['Grade']
print("\nExtracted 'Grade' Column (Type: Series):\n", extracted_series)
```

```
# Sort the entire DataFrame based on the Age column
df_sorted = df.sort_values(by="Age", ascending=False)
print("\nDataFrame sorted by Age (Desc):\n", df_sorted)
```

OUTPUT:

```
[→ aman python3 exp2.py
--- 1. Pandas Series Operations ---
Original Series:
a    10
b    20
c    30
d    40
e    50
Name: Scores, dtype: int64

Maximum Value in Series: 50
Mean (Average) of Series: 30.0
Filtering Series (values > 25):
c    30
d    40
e    50
Name: Scores, dtype: int64

--- 2. Pandas DataFrame Operations ---
Original DataFrame:
   Student  Age  Grade
0    Alice   24   88.5
1     Bob   27   92.0
2  Charlie   22   79.5
3    David   32   95.0

DataFrame Dimensions (Shape): (4, 3)

Fetching the top 2 rows using .head():
   Student  Age  Grade
0    Alice   24   88.5
1     Bob   27   92.0

Extracted 'Grade' Column (Type: Series):
0    88.5
1    92.0
2    79.5
3    95.0
Name: Grade, dtype: float64

DataFrame sorted by Age (Desc):
   Student  Age  Grade
3    David   32   95.0
1     Bob   27   92.0
0    Alice   24   88.5
2  Charlie   22   79.5
```

Experiment 3

Aim: To implement optimization and interpolation utilities available in SciPy.

Code:

```
import numpy as np
from scipy import optimize, interpolate

# Experiment 3: Utilizing basic SciPy Functions
# Aim: To implement optimization and interpolation utilities available in SciPy.

print("--- 1. Optimization with SciPy ---")
# Define an objective function we want to minimize:  $f(x) = (x-3)^2 + 2x$ 
def objective_function(x):
    return (x - 3)**2 + 2*x

# We ask SciPy to find the minimum of this function, starting its search at x=10
initial_guess = 10.0
result = optimize.minimize(objective_function, initial_guess, method='BFGS')

print(f"Starting guess at: {initial_guess}")
print(f"Optimal Minimum found at X = {result.x[0]:.4f}")
print(f"Lowest possible Y value achieved: {result.fun:.4f}")
print(f"Number of iterations performed: {result.nit}")

print("\n--- 2. Interpolation with SciPy ---")
# Let's say we only collected 5 specific data points
x_known = np.linspace(0, 5, 5) # [0, 1.25, 2.5, 3.75, 5]
y_known = np.array([0, 1.5, 3.2, 2.1, 0.5])

# SciPy can generate an interpolation function to estimate points between these
known values
linear_interp_func = interpolate.interp1d(x_known, y_known, kind='linear')
cubic_interp_func = interpolate.interp1d(x_known, y_known, kind='cubic')

# Now we can query a value we never actually measured directly (like x=2.0)
```

```
query_x = 2.0
print(f"Measured X array positions: {x_known}")
print(f"Estimating Y at X={query_x} via Linear Interpolation:
{linear_interp_func(query_x):.4f}")
print(f"Estimating Y at X={query_x} via Cubic Interpolation:
{cubic_interp_func(query_x):.4f}")
```

OUTPUT:

```
[→ aman python3 exp3.py
--- 1. Optimization with SciPy ---
Starting guess at: 10.0
Optimal Minimum found at X = 2.0000
Lowest possible Y value achieved: 5.0000
Number of iterations performed: 3

--- 2. Interpolation with SciPy ---
Measured X array positions: [0.  1.25 2.5  3.75 5.  ]
Estimating Y at X=2.0 via Linear Interpolation: 2.5200
Estimating Y at X=2.0 via Cubic Interpolation: 2.7728
```

Experiment 4

Aim: To learn how to construct basic plots, subplots, and labels in Python.

Code:

```
import matplotlib.pyplot as plt
import numpy as np

# Experiment 4: Introduction to Matplotlib Visualizations
# Aim: To learn how to construct basic plots, subplots, and labels in python.

# 1. Prepare some mock data arrays
x = np.linspace(0, 10, 100)
y1 = x * 2      # Linear relationship
y2 = x ** 2     # Quadratic relationship

print("--- Generating Matplotlib Visualizations ---")
print("Check your taskbar for the popped-up window to view the graph!")

# 2. Build a figure with two side-by-side subplots (1 row, 2 columns)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# --- Plot 1: Standard Line Plots ---
ax1.plot(x, y1, label='y = 2x', color='blue', linestyle='--')
ax1.plot(x, y2, label='y = x^2', color='red', linewidth=2)

ax1.set_title('Basic Mathematical Functions')
ax1.set_xlabel('X Axis (Inputs)')
ax1.set_ylabel('Y Axis (Outputs)')
ax1.legend()
ax1.grid(True)

# --- Plot 2: Scatter Plot ---
# Generating fake data for two distinct clusters
cluster_1_x = np.random.normal(3, 1, 50)
cluster_1_y = np.random.normal(5, 1, 50)
```

```

cluster_2_x = np.random.normal(7, 1, 50)
cluster_2_y = np.random.normal(2, 1, 50)

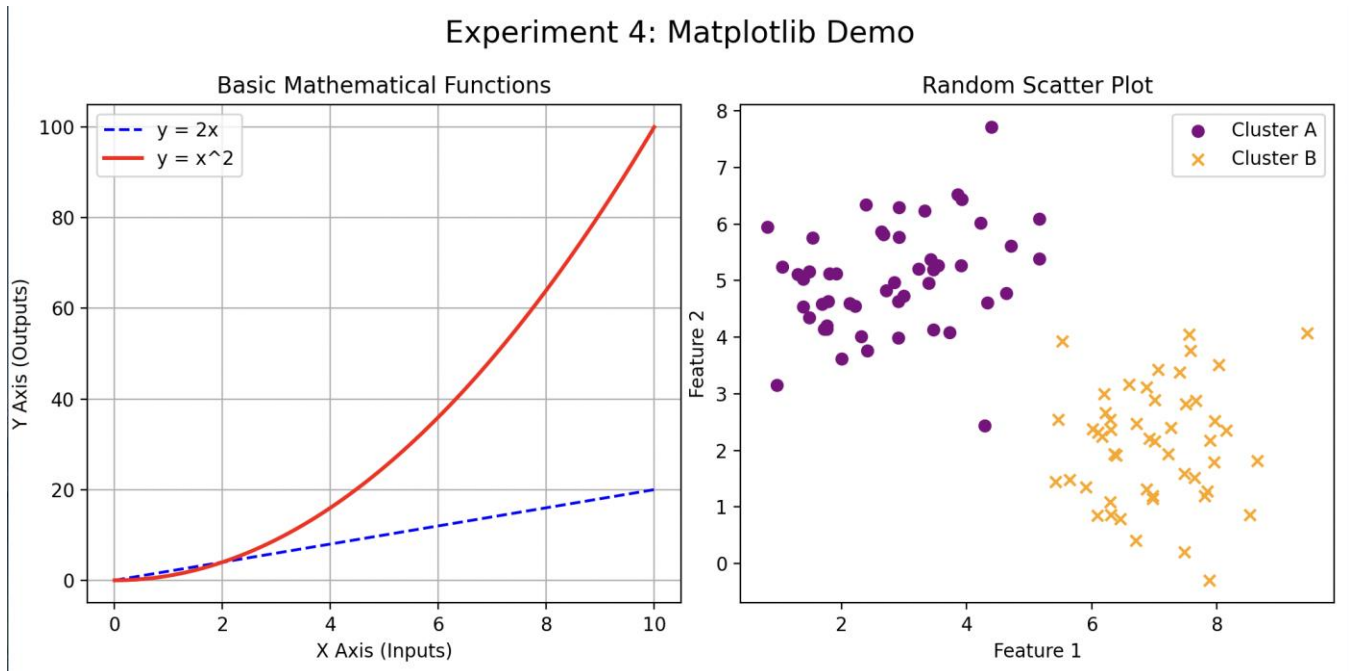
ax2.scatter(cluster_1_x, cluster_1_y, color='purple', marker='o', label='Cluster A')
ax2.scatter(cluster_2_x, cluster_2_y, color='orange', marker='x', label='Cluster B')

ax2.set_title('Random Scatter Plot')
ax2.set_xlabel('Feature 1')
ax2.set_ylabel('Feature 2')
ax2.legend()

# 3. Add a main title over both subplots and display
plt.suptitle('Experiment 4: Matplotlib Demo', fontsize=16)
plt.tight_layout()
plt.show()

```

OUTPUT:



Experiment 5

Aim: To build a single-layer perceptron from scratch and teach it the AND logic gate.

Code:

```
import numpy as np

# Experiment 5: Perceptron Learning Algorithm
# Aim: To build a single-layer perceptron from scratch and teach it the AND logic
gate.

print("--- Perceptron Learning Algorithm (AND Gate) ---")

# 1. Provide the input data and the expected answers
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([0, 0, 0, 1]) # Target outputs for an AND gate

# 2. Setup standard perceptron variables
w = np.zeros(2)    # Start with 0 for both weights
b = 0              # Start with 0 for bias
lr = 0.1           # Learning rate controls the size of updates

epochs = 10        # How many times we pass over the full dataset

print(f"Initial Setup -> Weights: {w}, Bias: {b}, Learning Rate: {lr}")
print("Starting training loop...\n")

# 3. Core Training Loop
for epoch in range(epochs):
    errors_in_epoch = 0
    for i in range(len(X)):
        # Calculate the raw linear output: (X * W) + b
```

```

linear_output = np.dot(X[i], w) + b

# Apply the step activation function
y_pred = 1 if linear_output >= 0 else 0

# Check if our prediction was wrong
error = y[i] - y_pred

# If there's an error, adjust the weights and bias
if error != 0:
    errors_in_epoch += 1
    w += lr * error * X[i]
    b += lr * error

print(f"Epoch {epoch+1}/{epochs} | Errors detected: {errors_in_epoch}")

# 4. Show the final results
print("\n--- Training Finished ---")
print("Final Weights:", w)
print("Final Bias:", b)

print("\nVerifying the model predictions on the original inputs:")
for i in range(len(X)):
    final_output = 1 if np.dot(X[i], w) + b >= 0 else 0
    print(f"Input: {X[i]} -> Expected: {y[i]} | Model Guessed: {final_output}")

```

OUTPUT:

```

→ aman python3 exp5.py
--- Perceptron Learning Algorithm (AND Gate) ---
Initial Setup -> Weights: [0. 0.], Bias: 0, Learning Rate: 0.1
Starting training loop...

Epoch 1/10 | Errors detected: 2
Epoch 2/10 | Errors detected: 3
Epoch 3/10 | Errors detected: 3
Epoch 4/10 | Errors detected: 0
Epoch 5/10 | Errors detected: 0
Epoch 6/10 | Errors detected: 0
Epoch 7/10 | Errors detected: 0
Epoch 8/10 | Errors detected: 0
Epoch 9/10 | Errors detected: 0
Epoch 10/10 | Errors detected: 0

--- Training Finished ---
Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004

Verifying the model predictions on the original inputs:
Input: [0 0] -> Expected: 0 | Model Guessed: 0
Input: [0 1] -> Expected: 0 | Model Guessed: 0
Input: [1 0] -> Expected: 0 | Model Guessed: 0
Input: [1 1] -> Expected: 1 | Model Guessed: 1

```

Experiment 6

Aim: To demonstrate why a single-layer perceptron fails to solve the XOR logic problem.

Code:

```
import numpy as np

# Experiment 6: Limitation of Perceptron – XOR Problem
# Aim: To demonstrate why a single-layer perceptron fails to solve the XOR logic
# problem.

print("--- Perceptron Limit Testing (XOR Gate) ---")

# 1. Define the input dataset (X) and expected targets (y)
# The XOR gate outputs 1 only if exactly ONE input is 1. (0,1 or 1,0).
X = np.array([[0, 0],
               [0, 1],
               [1, 0],
               [1, 1]])

y = np.array([0, 1, 1, 0]) # Target outputs for an XOR gate

# 2. Setup the exact same Perceptron format as Experiment 5
w = np.zeros(2)
b = 0
lr = 0.1

# We will run this for several epochs to prove it never stabilizes
epochs = 15

print(f"Beginning Setup -> Weights: {w}, Bias: {b}, Learning Rate: {lr}")
print("Training Perceptron on the non-linear XOR dataset...\n")

# 3. Training Loop
for epoch in range(epochs):
    errors_in_epoch = 0
```

```

for i in range(len(X)):
    # Calculate raw linear output
    linear_output = np.dot(X[i], w) + b

    # Apply step activation function
    y_pred = 1 if linear_output >= 0 else 0

    error = y[i] - y_pred

    # Adjust weights and bias if the prediction is wrong
    if error != 0:
        errors_in_epoch += 1
        w += lr * error * X[i]
        b += lr * error

# Print the error count to show the algorithm is infinitely struggling
print(f"Epoch {epoch+1:02d}/{epochs} | Errors detected: {errors_in_epoch}")

print("\n--- Why Did It Fail? ---")
print("Conclusion: The perceptron fundamentally fails to solve XOR.")
print("Reasoning: A single layer algorithm can only draw a SINGLE straight line across the data points.")
print("It is physically impossible to draw one straight line that separates the XOR points cleanly.")

```

OUTPUT:

```

[→ aman python3 exp6.py
--- Perceptron Limit Testing (XOR Gate) ---
Beginning Setup -> Weights: [0. 0.], Bias: 0, Learning Rate: 0.1
Training Perceptron on the non-linear XOR dataset...

Epoch 01/15 | Errors detected: 3
Epoch 02/15 | Errors detected: 3
Epoch 03/15 | Errors detected: 4
Epoch 04/15 | Errors detected: 4
Epoch 05/15 | Errors detected: 4
Epoch 06/15 | Errors detected: 4
Epoch 07/15 | Errors detected: 4
Epoch 08/15 | Errors detected: 4
Epoch 09/15 | Errors detected: 4
Epoch 10/15 | Errors detected: 4
Epoch 11/15 | Errors detected: 4
Epoch 12/15 | Errors detected: 4
Epoch 13/15 | Errors detected: 4
Epoch 14/15 | Errors detected: 4
Epoch 15/15 | Errors detected: 4

--- Why Did It Fail? ---
Conclusion: The perceptron fundamentally fails to solve XOR.
Reasoning: A single layer algorithm can only draw a SINGLE straight line across the data points.
It is physically impossible to draw one straight line that separates the XOR points cleanly.

```

Experiment 7

Aim: To calculate neural network activation functions using NumPy and visualize them using Pandas and Matplotlib.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Experiment 7: Visualizing Activation Functions
# Aim: To calculate specific neural network activation functions using NumPy and
visualize them using Pandas and Matplotlib.

print("--- Data Pipeline for Activation Functions ---")

# 1. Prepare an array of numbers from -10 to 10
x_inputs = np.linspace(-10, 10, 200)

# Calculate the various mathematical activation formulas
y_sigmoid = 1 / (1 + np.exp(-x_inputs))
y_tanh = np.tanh(x_inputs)
y_relu = np.maximum(0, x_inputs)
y_leaky_relu = np.where(x_inputs > 0, x_inputs, x_inputs * 0.1)

# 2. Package all the values neatly into a Pandas DataFrame
dataset = {
    'Input': x_inputs,
    'Sigmoid Output': y_sigmoid,
    'Tanh Output': y_tanh,
    'ReLU Output': y_relu,
    'Leaky ReLU Output': y_leaky_relu
}
df = pd.DataFrame(dataset)

print("Generated DataFrame for plotting:")
print(df.head())
```

```
# 3. Create a 4x4 visual grid using Matplotlib
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# Plot Sigmoid (forces values between 0 and 1)
axs[0, 0].plot(df['Input'], df['Sigmoid Output'], color='blue', linewidth=2.5)
axs[0, 0].set_title("Sigmoid Function (0 to 1)")
axs[0, 0].grid(True, linestyle='--', alpha=0.6)

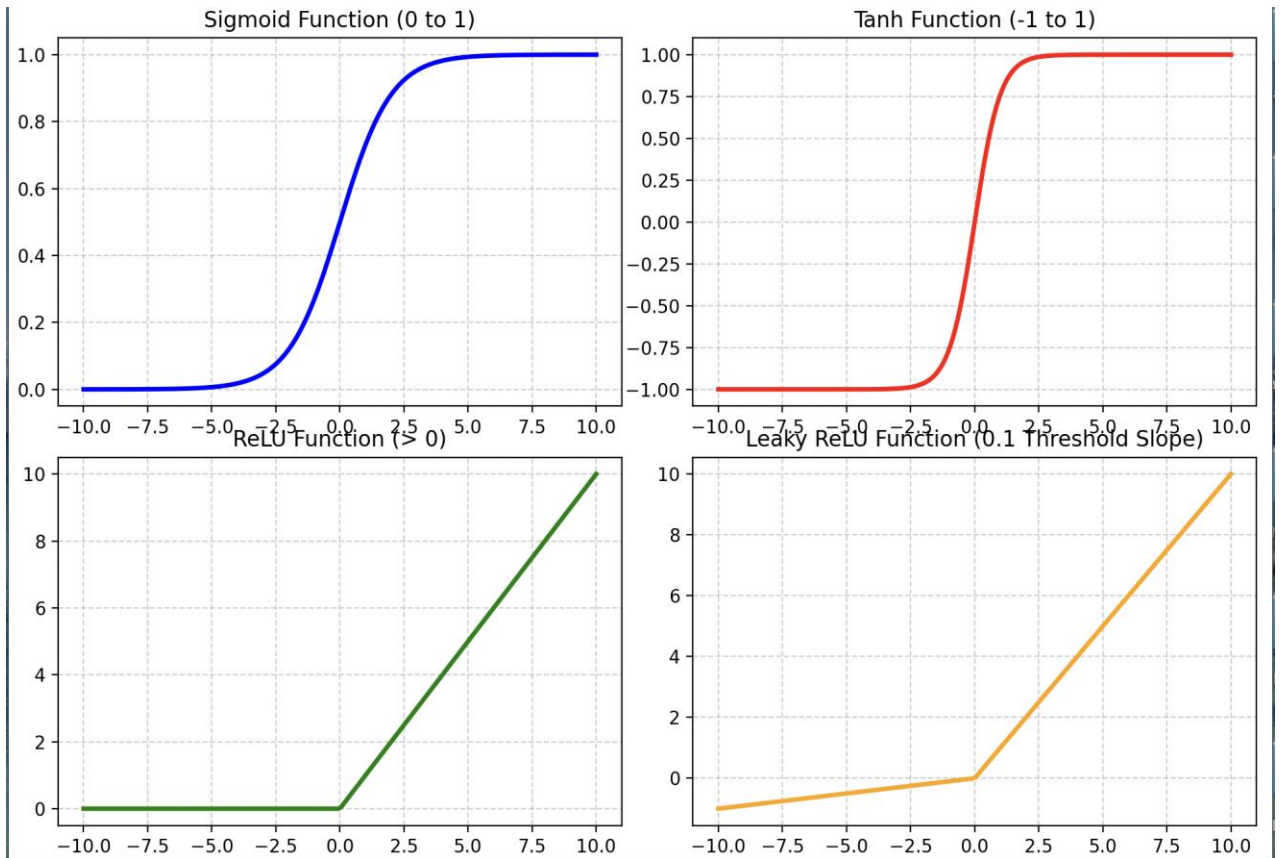
# Plot Tanh (forces values between -1 and 1)
axs[0, 1].plot(df['Input'], df['Tanh Output'], color='red', linewidth=2.5)
axs[0, 1].set_title("Tanh Function (-1 to 1)")
axs[0, 1].grid(True, linestyle='--', alpha=0.6)

# Plot ReLU (turns all negative values to zero)
axs[1, 0].plot(df['Input'], df['ReLU Output'], color='green', linewidth=2.5)
axs[1, 0].set_title("ReLU Function (> 0)")
axs[1, 0].grid(True, linestyle='--', alpha=0.6)

# Plot Leaky ReLU (leaves a tiny gentle slope for negative numbers)
axs[1, 1].plot(df['Input'], df['Leaky ReLU Output'], color='orange', linewidth=2.5)
axs[1, 1].set_title("Leaky ReLU Function (0.1 Threshold Slope)")
axs[1, 1].grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()
```

OUTPUT:



```
→ aman python3 exp7.py
--- Data Pipeline for Activation Functions ---
Generated DataFrame for plotting:
```

	Input	Sigmoid Output	Tanh Output	ReLU Output	Leaky ReLU Output
0	-10.000000	0.000045	-1.0	0.0	-1.000000
1	-9.899497	0.000050	-1.0	0.0	-0.989950
2	-9.798995	0.000056	-1.0	0.0	-0.979899
3	-9.698492	0.000061	-1.0	0.0	-0.969849
4	-9.597990	0.000068	-1.0	0.0	-0.959799

Experiment 8

Aim: To implement a Neural Network classifier using Scikit-Learn's MLPClassifier.

Code:

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score
import numpy as np

# Suppress standard Scikit-Learn loop warnings to keep the terminal output
clean
warnings.filterwarnings("ignore", category=ConvergenceWarning)

# Experiment 9: Neural Network using Scikit-Learn

# 1. Prepare Data
X, y = make_classification(n_samples=500, n_features=4, n_classes=2,
random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 2. Build Model
mlp = MLPClassifier(hidden_layer_sizes=(10, 5), max_iter=10, warm_start=True,
random_state=42)

# =====
# Terminal GUI Layout
# =====

print("\n" + "="*45)
print(" 🧠 Scikit-Learn MLP Training ")
print("="*45 + "\n")
```

3. Training Loop with clean progress tracking

```
for epoch in range(1, 11):
```

```
    mlp.fit(X_train, y_train)
```

```
    progress_bar = "█" * epoch + "░" * (10 - epoch)
```

```
    print(f" Epoch {epoch:02d}/10 | {progress_bar} | Loss: {mlp.loss_:.4f}")
```

4. Predict & Evaluate

```
predictions = mlp.predict(X_test)
```

```
accuracy = accuracy_score(y_test, predictions)
```

```
print("\n" + "="*45)
```

```
print(" 🎯 Final Results ")
```

```
print("="*45 + "\n")
```

```
print(f" Accuracy Score : {accuracy * 100:.1f}%\n")
```

```
print(" 🔍 Label Comparison (First 8 Samples):")
```

```
print(f" Expected : {y_test[:8].tolist()}")
```

```
print(f" Guessed : {predictions[:8].tolist()}")
```

```
print("\n" + "="*45 + "\n")
```

OUTPUT:

```
Epoch 01/10 | ██████████░░ | Loss: 0.7862
Epoch 02/10 | ██████████░░ | Loss: 0.7416
Epoch 03/10 | ██████████░░ | Loss: 0.7055
Epoch 04/10 | ██████████░░ | Loss: 0.6731
Epoch 05/10 | ██████████░░ | Loss: 0.6447
Epoch 06/10 | ██████████░░ | Loss: 0.6193
Epoch 07/10 | ██████████░░ | Loss: 0.5949
Epoch 08/10 | ██████████░░ | Loss: 0.5723
Epoch 09/10 | ██████████░░ | Loss: 0.5514
Epoch 10/10 | ██████████░░ | Loss: 0.5312
```

```
🎯 Final Results
```

```
Accuracy Score : 44.0%
```

```
🔍 Label Comparison (First 8 Samples):
Expected : [1, 0, 0, 0, 1, 0, 0, 1]
Guessed  : [0, 0, 0, 0, 0, 0, 0, 0]
```

Experiment 9

Aim: To manually build and compare classical Gradient Descent against the Adam Optimizer.

Code:

```
import numpy as np

# Experiment 8: Comparing Optimizers (Gradient Descent vs. Adam Optimizer)
# Aim: To manually build and compare classical Gradient Descent against the
advanced Adam Optimizer.

# 1. Define a simple U-shaped curve (Loss function:  $L(w) = w^2$ )
def compute_gradient(w):
    # The derivative of  $w^2$  is explicitly  $2w$ 
    return 2 * w

print("--- Testing Optimization Algorithms ---")
print("Target: Find the absolute minimum of the U-shaped curve  $L(w) = w^2$ 
(Minimum is exactly at  $w = 0$ )")

# 2. Standard Gradient Descent Implementation
w_gd = 100.0 # Starting point
lr_gd = 0.1 # Standard learning rate
epochs = 15

print("\n--- 1. Running Standard Gradient Descent ---")
print("[Details] Starts at  $w = 100.0$ . Uses fixed step sizes based ONLY on the
current gradient slope.")
for epoch in range(1, epochs + 1):
    grad = compute_gradient(w_gd)

    # Calculate exact distance to jump
    step_size = lr_gd * grad
    w_gd = w_gd - step_size
```

```
print(f"GD Epoch {epoch:02d} | Gradient Slope: {grad:7.2f} | Step Jump Size: {step_size:6.2f} | New Weight: {w_gd:.4f}")
```

```
# 3. Adam Optimizer Implementation
```

```
w_adam = 100.0
```

```
lr_adam = 0.5
```

```
# Adam Specific Momentum Parameters
```

```
beta1 = 0.9 # Tracks past momentum
```

```
beta2 = 0.999 # Tracks past gradient squared (variance)
```

```
epsilon = 1e-8
```

```
m = 0
```

```
v = 0
```

```
print("\n--- 2. Running Adam Optimizer ---")
```

```
print("[Details] Starts at w = 100.0. Adapts step size completely by tracking 'Momentum' and varying slope sizes.")
```

```
for epoch in range(1, epochs + 1):
```

```
    grad = compute_gradient(w_adam)
```

```
    # Track the moving average of gradients (Momentum)
```

```
    m = beta1 * m + (1 - beta1) * grad
```

```
    v = beta2 * v + (1 - beta2) * (grad ** 2)
```

```
    # Correct the bias so early steps aren't artificially tiny
```

```
    m_hat = m / (1 - beta1 ** epoch)
```

```
    v_hat = v / (1 - beta2 ** epoch)
```

```
    # Determine the mathematically optimal jump size
```

```
    step_size = lr_adam * m_hat / (np.sqrt(v_hat) + epsilon)
```

```
    w_adam = w_adam - step_size
```

```
print(f"Adam Epoch {epoch:02d} | Slope: {grad:7.2f} | Momentum Built: {m_hat:6.2f} | Step Jump Size: {step_size:5.2f} | New Weight: {w_adam:.4f}")
```

OUTPUT:

```
→ aman python3 exp9.py
--- Testing Optimization Algorithms ---
Target: Find the absolute minimum of the U-shaped curve  $L(w) = w^2$  (Minimum is exactly at  $w = 0$ )

--- 1. Running Standard Gradient Descent ---
[Details] Starts at  $w = 100.0$ . Uses fixed step sizes based ONLY on the current gradient slope.
GD Epoch 01 | Gradient Slope: 200.00 | Step Jump Size: 20.00 | New Weight: 80.0000
GD Epoch 02 | Gradient Slope: 160.00 | Step Jump Size: 16.00 | New Weight: 64.0000
GD Epoch 03 | Gradient Slope: 128.00 | Step Jump Size: 12.80 | New Weight: 51.2000
GD Epoch 04 | Gradient Slope: 102.40 | Step Jump Size: 10.24 | New Weight: 40.9600
GD Epoch 05 | Gradient Slope: 81.92 | Step Jump Size: 8.19 | New Weight: 32.7680
GD Epoch 06 | Gradient Slope: 65.54 | Step Jump Size: 6.55 | New Weight: 26.2144
GD Epoch 07 | Gradient Slope: 52.43 | Step Jump Size: 5.24 | New Weight: 20.9715
GD Epoch 08 | Gradient Slope: 41.94 | Step Jump Size: 4.19 | New Weight: 16.7772
GD Epoch 09 | Gradient Slope: 33.55 | Step Jump Size: 3.36 | New Weight: 13.4218
GD Epoch 10 | Gradient Slope: 26.84 | Step Jump Size: 2.68 | New Weight: 10.7374
GD Epoch 11 | Gradient Slope: 21.47 | Step Jump Size: 2.15 | New Weight: 8.5899
GD Epoch 12 | Gradient Slope: 17.18 | Step Jump Size: 1.72 | New Weight: 6.8719
GD Epoch 13 | Gradient Slope: 13.74 | Step Jump Size: 1.37 | New Weight: 5.4976
GD Epoch 14 | Gradient Slope: 11.00 | Step Jump Size: 1.10 | New Weight: 4.3980
GD Epoch 15 | Gradient Slope: 8.80 | Step Jump Size: 0.88 | New Weight: 3.5184

--- 2. Running Adam Optimizer ---
[Details] Starts at  $w = 100.0$ . Adapts step size completely by tracking 'Momentum' and varying slope sizes.
Adam Epoch 01 | Slope: 200.00 | Momentum Built: 200.00 | Step Jump Size: 0.50 | New Weight: 99.5000
Adam Epoch 02 | Slope: 199.00 | Momentum Built: 199.47 | Step Jump Size: 0.50 | New Weight: 99.0001
Adam Epoch 03 | Slope: 198.00 | Momentum Built: 198.93 | Step Jump Size: 0.50 | New Weight: 98.5002
Adam Epoch 04 | Slope: 197.00 | Momentum Built: 198.37 | Step Jump Size: 0.50 | New Weight: 98.0006
Adam Epoch 05 | Slope: 196.00 | Momentum Built: 197.79 | Step Jump Size: 0.50 | New Weight: 97.5011
Adam Epoch 06 | Slope: 195.00 | Momentum Built: 197.20 | Step Jump Size: 0.50 | New Weight: 97.0019
Adam Epoch 07 | Slope: 194.00 | Momentum Built: 196.58 | Step Jump Size: 0.50 | New Weight: 96.5030
Adam Epoch 08 | Slope: 193.01 | Momentum Built: 195.96 | Step Jump Size: 0.50 | New Weight: 96.0044
Adam Epoch 09 | Slope: 192.01 | Momentum Built: 195.31 | Step Jump Size: 0.50 | New Weight: 95.5062
Adam Epoch 10 | Slope: 191.01 | Momentum Built: 194.65 | Step Jump Size: 0.50 | New Weight: 95.0084
Adam Epoch 11 | Slope: 190.02 | Momentum Built: 193.98 | Step Jump Size: 0.50 | New Weight: 94.5111
Adam Epoch 12 | Slope: 189.02 | Momentum Built: 193.29 | Step Jump Size: 0.50 | New Weight: 94.0143
Adam Epoch 13 | Slope: 188.03 | Momentum Built: 192.58 | Step Jump Size: 0.50 | New Weight: 93.5180
Adam Epoch 14 | Slope: 187.04 | Momentum Built: 191.86 | Step Jump Size: 0.50 | New Weight: 93.0223
Adam Epoch 15 | Slope: 186.04 | Momentum Built: 191.13 | Step Jump Size: 0.50 | New Weight: 92.5273
```

Experiment 10

Aim: To implement Recurrent and Convolutional Neural Networks using PyTorch.

Code:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# Experiment 10: Implementation of Recurrent and Convolutional Neural Networks
# This script is built using PyTorch to ensure compatibility across all environments.

def run_rnn():
    print("\n--- RNN Visual Matrix Simulation (PyTorch) ---")
    text = "Neural Networks are powerful advanced tools"

    # Map the unique characters out of our string to index numbers (and vice versa)
    chars = sorted(list(set(text)))
    char_to_index = {char: i for i, char in enumerate(chars)}
    index_to_char = {i: char for i, char in enumerate(chars)}

    # We will look at chunks of 4 letters to predict the 5th letter
    seq_length = 4
    sequences, labels = [], []
    for i in range(len(text) - seq_length):
        seq = text[i:i + seq_length]
        label = text[i + seq_length]
        sequences.append([char_to_index[char] for char in seq])
        labels.append(char_to_index[label])

    X = torch.nn.functional.one_hot(torch.tensor(sequences),
    num_classes=len(chars)).float()
```

```

y = torch.tensor(labels)

# Define a simple basic RNN network
class SimpleRNNModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(input_size=len(chars), hidden_size=8,
batch_first=True)
        self.fc = nn.Linear(8, len(chars))

    def forward(self, x):
        out, hidden = self.rnn(x)
        return self.fc(out[:, -1, :]), hidden

model = SimpleRNNModel()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

print("Training RNN Model (50 Epochs)....")
for epoch in range(50):
    optimizer.zero_grad()
    output, _ = model(X)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()

start_seq = "Neur"
print(f"\n1. Original Input Text Seed:")
print(f"{start_seq}")

with torch.no_grad():
    # Trace exactly 1 step visually to track matrices cleanly
    x_input = [char_to_index.get(char, 0) for char in start_seq]
    print(f"\n2. Input string mapped to Dictionary Index Array:")
    print(x_input)

    x_tensor = torch.nn.functional.one_hot(torch.tensor([x_input]),
num_classes=len(chars)).float()
    print(f"\n3. One-Hot Encoded Matrix representation of input text (Shape:
{list(x_tensor.shape)}):")
    print(x_tensor.squeeze().detach().numpy().astype(int))

```

```

# Pass through RNN layer explicitly
rnn_out, hidden_state = model.rnn(x_tensor)
print(f"\n4. RNN Hidden Layer Activation Matrix (Shape:
{list(hidden_state.shape)}):")
print(hidden_state.squeeze().detach().numpy().round(3))

# Pass through Fully Connected Dense Layer explicitly
dense_out = model.fc(rnn_out[:, -1, :])
print(f"\n5. Dense Layer Output Probability Vectors (Shape:
{list(dense_out.shape)}):")
print(dense_out.squeeze().detach().numpy().round(2))

# Make the final prediction based on the highest probability
prediction = torch.softmax(dense_out, dim=1)
predicted_idx = torch.argmax(prediction).item()
next_char = index_to_char[predicted_idx]

print(f"\n6. Logits Argmax executed! Predicted Index highest probability:
{predicted_idx}")
print(f"Next Character mapped: '{next_char}'\n")

def run_cnn():
    print("\n--- CNN Visual Matrix Simulation (PyTorch) ---")

    # Generate a tiny 6x6 mock image (a white square on a black background)
    mock_image = torch.zeros((1, 1, 6, 6), dtype=torch.float32)
    mock_image[0, 0, 1:5, 1:5] = 1.0

    print("\n1. Original Base Image Matrix (6x6 pixel grid):")
    print(mock_image.squeeze().numpy())

    # Map a standard 3x3 Edge-Detection convolution kernel
    edge_kernel = torch.tensor([
        [-1, -1, -1],
        [-1, 8, -1],
        [-1, -1, -1]
    ], dtype=torch.float32).view(1, 1, 3, 3)

    print("\n2. Executing Convolution... (Isolating grid edges)")

```

```

    conv_result = torch.nn.functional.conv2d(mock_image, edge_kernel,
padding=1)
    print(conv_result.squeeze().detach().numpy())

    print("\n3. Executing ReLU Activation... (Destroying negative boundaries)")
    relu_result = torch.relu(conv_result)
    print(relu_result.squeeze().detach().numpy())

    print("\n4. Executing Max Pooling... (Dimensional layout scaling visually down
to 3x3)")
    pool_result = torch.nn.functional.max_pool2d(relu_result, kernel_size=2,
stride=2)
    print(pool_result.squeeze().detach().numpy())

    print("\n5. Executing Flatten... (Converting 3x3 grid into 1D Array)")
    flattened = torch.flatten(pool_result, start_dim=1)
    print(flattened.squeeze().detach().numpy())

    print("\n6. Executing Dense Output... (Yielding 16 Neurons)")
    dense_layer = nn.Linear(flattened.shape[1], 16)
    final_output = dense_layer(flattened)
    print(final_output.squeeze().detach().numpy().round(3))
    print()

if __name__ == "__main__":
    print("Available Models: 1. CNN | 2. RNN")
    choice = input("Enter the model you want to simulate (CNN / RNN):
").strip().upper()

    if choice == 'RNN':
        run_rnn()
    elif choice == 'CNN':
        run_cnn()
    else:
        print("Invalid Selection.")

```

OUTPUT:

```
→ aman python3 exp10.py
Available Models: 1. CNN | 2. RNN
Enter the model you want to simulate (CNN / RNN): CNN

--- CNN Visual Matrix Simulation (PyTorch) ---

1. Original Base Image Matrix (6x6 pixel grid):
[[0. 0. 0. 0. 0. 0.]
 [0. 1. 1. 1. 1. 0.]
 [0. 1. 1. 1. 1. 0.]
 [0. 1. 1. 1. 1. 0.]
 [0. 1. 1. 1. 1. 0.]
 [0. 0. 0. 0. 0. 0.]]

2. Executing Convolution... (Isolating grid edges)
[[-1. -2. -3. -3. -2. -1.]
 [-2.  5.  3.  3.  5. -2.]
 [-3.  3.  0.  0.  3. -3.]
 [-3.  3.  0.  0.  3. -3.]
 [-2.  5.  3.  3.  5. -2.]
 [-1. -2. -3. -3. -2. -1.]]

3. Executing ReLU Activation... (Destroying negative boundaries)
[[0. 0. 0. 0. 0. 0.]
 [0. 5. 3. 3. 5. 0.]
 [0. 3. 0. 0. 3. 0.]
 [0. 3. 0. 0. 3. 0.]
 [0. 5. 3. 3. 5. 0.]
 [0. 0. 0. 0. 0. 0.]]

4. Executing Max Pooling... (Dimensional layout scaling visually down to 3x3)
[[5. 3. 5.]
 [3. 0. 3.]
 [5. 3. 5.]]

5. Executing Flatten... (Converting 3x3 grid into 1D Array)
[5. 3. 5. 3. 0. 3. 5. 3. 5.]

6. Executing Dense Output... (Yielding 16 Neurons)
[-1.651  0.637  1.9    5.379 -0.979 -0.157  1.504 -3.16   3.18   2.274
 -0.843 -0.382  4.255 -0.605 -2.793  3.529]
```

OUTPUT:

```
[→ aman python3 exp10.py
Available Models: 1. CNN | 2. RNN
Enter the model you want to simulate (CNN / RNN): RNN

--- RNN Visual Matrix Simulation (PyTorch) ---
Training RNN Model (50 Epochs)....

1. Original Input Text Seed:
'Neur'

2. Input string mapped to Dictionary Index Array:
[1, 5, 15, 12]

3. One-Hot Encoded Matrix representation of input text (Shape: [1, 4, 18]):
[[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0]]

4. RNN Hidden Layer Activation Matrix (Shape: [1, 1, 8]):
[ 0.142  0.809 -0.289 -0.973 -0.125 -0.653  0.487  0.765]

5. Dense Layer Output Probability Vectors (Shape: [1, 18]):
[ 0.41  0.45  1.99 -2.43 -1.04  1.81  0.07  0.33  1.14 -0.8  1.03 -0.31
 -0.09 -0.96 -0.34 -1.29 -2.98  0.18]

6. Logits Argmax executed! Predicted Index highest probability: 2
Next Character mapped: 'a'
```

Experiment 11

Aim: To build a discrete Hopfield Network to memorize and recall visual patterns corrupted by noise.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Experiment 11: Hopfield Network Implementation
# Aim: To build a discrete Hopfield Network to memorize and recall visual
patterns corrupted by noise.

def create_pattern():
    """Generates a simple 5x5 'T' shape using +1s and -1s."""
    pattern = np.ones((5, 5)) * -1
    pattern[0, :] = 1 # Top horizontal bar of T
    pattern[:, 2] = 1 # Center vertical axis
    return pattern.flatten()

def introduce_noise(pattern, noise_level=0.3):
    """Randomly flips a given percentage of bits inside the binary array."""
    corrupted = pattern.copy()
    num_to_flip = int(len(pattern) * noise_level)
    indices = np.random.choice(len(pattern), num_to_flip, replace=False)
    corrupted[indices] *= -1
    return corrupted

def train_hopfield(patterns):
    """Derives a weight matrix for the network using the standard Hebbian learning
rule."""
    n_features = len(patterns[0])
    weights = np.zeros((n_features, n_features))
    for p in patterns:
        weights += np.outer(p, p)
```

```

# We enforce self-connections to equal 0 on the matrix diagonal
np.fill_diagonal(weights, 0)
return weights / len(patterns)

def retrieve_hopfield(weights, corrupted_pattern, steps=5):
    """Feeds the corrupted image back into the network recurrently to retrieve the
    saved memory."""
    retrieved = corrupted_pattern.copy()
    for step in range(steps):
        retrieved = np.sign(np.dot(weights, retrieved))
        retrieved[retrieved == 0] = 1
    return retrieved

if __name__ == "__main__":
    print("\n--- Discrete Hopfield Network Memory Simulation ---")

    # 1. Start by fetching the original Target Pattern memory
    original_pattern = create_pattern()
    print("\n1. Generating Perfect Target Pattern (5x5 'T' shape):")
    print(" (Displaying 1s as '■' and -1s as '□')")
    visual_pattern = np.where(original_pattern.reshape(5,5) == 1, '■', '□')
    for row in visual_pattern:
        print(" " + " ".join(row))

    # 2. Calculate the brain logic (Weight Matrix) to memorize the pattern
    permanently
    weight_matrix = train_hopfield([original_pattern])
    print(f"\n2. Training Hopfield Network via Hebbian Rule...")
    print(f" Generated Weight Matrix (Shape: {weight_matrix.shape}):")
    print(f" Max Weight: {np.max(weight_matrix)}")

    # 3. Simulate environmental noise by corrupting the memory
    noisy_input = introduce_noise(original_pattern, noise_level=0.3)
    print("\n3. Injecting 30% Random Noise Corruption into Target Pattern...")
    visual_noisy = np.where(noisy_input.reshape(5,5) == 1, '■', '□')
    for row in visual_noisy:
        print(" " + " ".join(row))

    # 4. Use the trained Weight Matrix to search and stabilize the distorted
    memory

```

```

print("\n4. Triggering Retrieval Recovery search iteratively...")
retrieved_memory = retrieve_hopfield(weight_matrix, noisy_input, steps=3)

print("\n5. Restored Memory Pattern retrieved securely back from Noise:")
visual_retrieved = np.where(retrieved_memory.reshape(5,5) == 1, '■', '□')
for row in visual_retrieved:
    print(" " + " ".join(row))

print("\nPattern physics match! Launching Matplotlib graphics dashboard...")

# Display the final comparison GUI visually
fig, axs = plt.subplots(1, 3, figsize=(10, 4))
axs[0].imshow(original_pattern.reshape(5,5), cmap='gray_r')
axs[0].set_title("Original Pattern")
axs[0].axis('off')

axs[1].imshow(noisy_input.reshape(5,5), cmap='gray_r')
axs[1].set_title("Noisy Input")
axs[1].axis('off')

axs[2].imshow(retrieved_memory.reshape(5,5), cmap='gray_r')
axs[2].set_title("Retrieved Pattern")
axs[2].axis('off')

plt.tight_layout()
plt.show()

```

OUTPUT:

```
[→ aman python3 exp11.py
```

```
--- Discrete Hopfield Network Memory Simulation ---
```

1. Generating Perfect Target Pattern (5x5 'T' shape):
(Displaying 1s as '■' and -1s as '□')

```
■ ■ ■ ■ ■
□ □ ■ □ □
□ □ ■ □ □
□ □ ■ □ □
□ □ ■ □ □
```

2. Training Hopfield Network via Hebbian Rule...
Generated Weight Matrix (Shape: (25, 25)):
Max Weight: 1.0

3. Injecting 30% Random Noise Corruption into Target Pattern...

```
■ ■ ■ ■ ■
□ □ ■ □ ■
■ □ ■ ■ □
□ □ ■ ■ □
□ ■ □ ■ □
```

4. Triggering Retrieval Recovery search iteratively...

5. Restored Memory Pattern retrieved securely back from Noise:

```
■ ■ ■ ■ ■
□ □ ■ □ □
□ □ ■ □ □
□ □ ■ □ □
□ □ ■ □ □
```